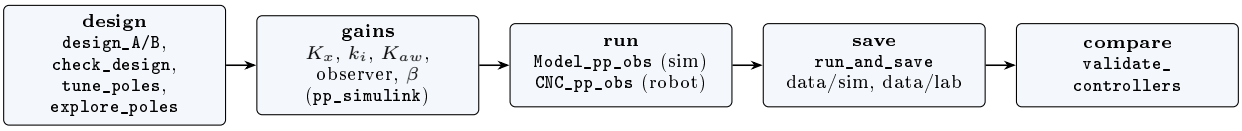
Pole placement for the CNC plotter

theory behind the `pp_design` folder: model, controller in Simulink, parameters, requirements, search, trajectory and evaluation; disturbances and voltage at the end

1 The workflow at a glance



The design scripts choose three closed-loop poles, compute the gains and write them (and every variable of `scripts/parameters.m`) to the workspace. The Simulink model runs the square path; the run is saved and compared with other runs (PID, other designs, simulation vs robot) by `validate_controllers.m`.

2 Plant and control law

Motor model (one per joint, identified with the linkage attached)

Angle in encoder degrees, voltage u, load d as an equivalent voltage:

$$\tau\ddot{\theta} + \dot{\theta} = b\,(u + d), \qquad G(s) = \frac{b}{s(\tau s + 1)}, \qquad b = -\tfrac{180}{\pi}k, \qquad x = \begin{bmatrix}\theta \\ \omega\end{bmatrix},\ A = \begin{bmatrix}0 & 1 \\ 0 & -\frac{1}{\tau}\end{bmatrix},\ B = \begin{bmatrix}0 \\ \frac{b}{\tau}\end{bmatrix},\ C = [1\ 0].$$

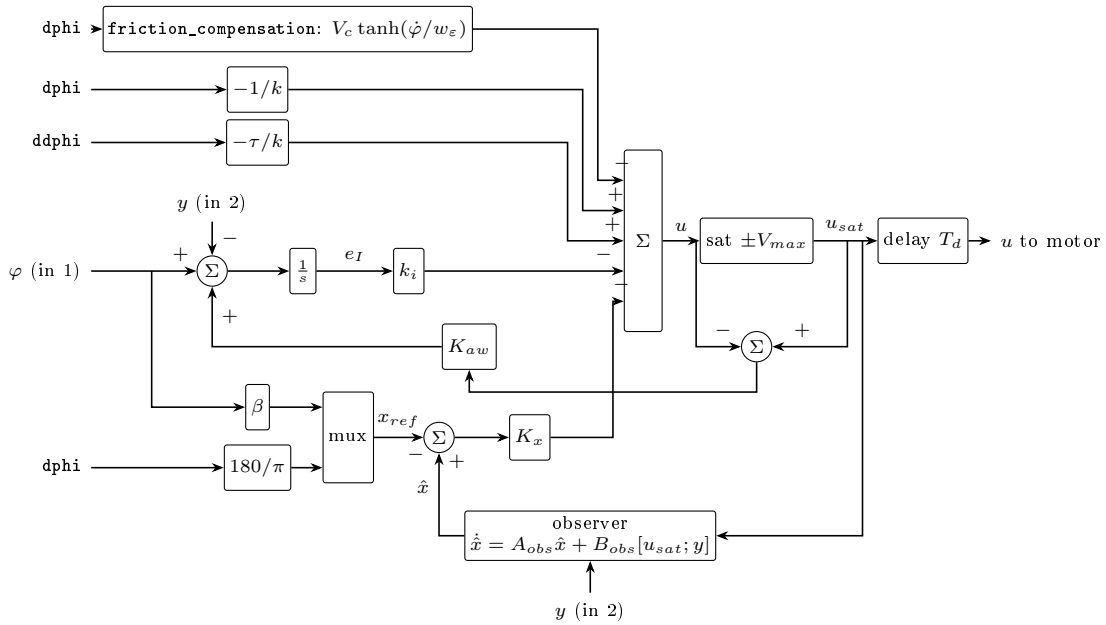
k, τ = `k_xxxx_load`, `tau_xxxx_load` (`parameters.m`, section 4). The minus sign in b is the encoder convention: a positive voltage gives a negative measured angle. $1/\tau \approx 27\,\mathrm{rad/s}$ is the mechanical pole; $|b| \approx 82\,^\circ/(\mathrm{V\,s})$: 1 V turns the joint at about $82^\circ/\mathrm{s}$ in steady state.

Control law (the same in the scripts and in Simulink)

$$u = \underbrace{-k_1(\hat{\theta} - \beta r) - k_2(\hat{\omega} - \dot{r})}_{\text{state feedback}} \underbrace{-\,k_i e_I}_{\text{integral}} \underbrace{+\,\frac{\dot{r} + \tau\ddot{r}}{b}}_{\text{feedforward}} \underbrace{-\,V_c \tanh(\dot{\varphi}/w_\varepsilon)}_{\text{friction comp.}}, \qquad \dot{e}_I = r - \theta + K_{aw}(u_{sat} - u).$$

Only θ is measured: a Luenberger observer gives $\hat{x} = [\hat{\theta}, \hat{\omega}]$. The integral removes constant errors, the feedforward gives the voltage the motion needs, β weights the reference, the anti-windup stops the integral when the amplifier saturates.

The controller block in Simulink (`pp + luenberg 3165`, the same for 3166)



Signs of the voltage sum (`-++-`): $-$friction compensation, $+$speed feedforward, $+$acceleration feedforward, $-k_i e_I$, $-K_x(\hat{x} - x_{ref})$. `To Workspace` blocks log u (`u_3165`) and u_{sat} (`usat_3165`); a scope shows u_{sat}.

Every block and why it is there

block	role
φ, `dphi`, `ddphi`	joint reference from the inverse kinematics: φ in degrees (after a 100°/s rate limiter), $\dot\varphi, \ddot\varphi$ in rad/s and rad/s^2 (not limited)
$-1/k$, $-\tau/k$	feedforward: the voltage the nominal motor needs to follow $\dot\varphi, \ddot\varphi$ (same as $(\dot r + \tau\ddot r)/b$ in degrees)
`friction_compensation`	$V_c \tanh(\dot\varphi/w_\varepsilon)$, entering with $-$: same sign as the speed feedforward, it adds the voltage that Coulomb friction takes away. V_c = `Vc_xxxx`, w_ε = `w_eps` (`parameters.m`, section 5); tanh instead of sign avoids a voltage jump at zero speed
Σ, $1/s$, k_i	integral of the error $r-\theta$ (+ anti-windup), gain k_i
β, $180/\pi$, mux	reference state $x_{ref} = [\beta\varphi;\ \dot\varphi \cdot 180/\pi]$ (degrees)
observer	$\dot{\hat x} = A_{obs}\hat x + B_{obs}[u_{sat}; y]$ with $A_{obs} = A - LC$, $B_{obs} = [B,\ L]$: it uses the voltage really applied (u_{sat})
K_x (`K*u`)	state feedback on $\hat x - x_{ref}$
saturation	$\pm V_{max} = \pm 6$ V: the amplifier limit
K_{aw}	anti-windup: $K_{aw}(u_{sat} - u)$ into the integrator (see below)
delay T_d	`Td_xxxx`: delay of the simulated motor (section 11); 0 when `use_real_plant = false`. It belongs to the plant: it should not be copied into `CNC_pp_obs.slx`

Saturation, anti-windup and delay

Saturation. Not linear, so the pole-placement formulas cannot include it. The design keeps the test step inside ± 6 V (requirement *peak voltage*); large transients (start of the path) may still saturate.
Anti-windup. While saturated, the integral would keep growing (windup) and later cause a large overshoot. With $K_{aw} = -\overline{|p|}/k_i$ (mean speed of the poles, `pp_simulink.m`) the term $K_{aw}(u_{sat} - u)$ moves the integral voltage $-k_i e_I$ towards the saturated value with time constant $1/\overline{|p|}$; when $u = u_{sat}$ it is zero and the loop is the linear one.
Delay of the digital controller ($T_d = 1.5\,T_s = 3$ ms, `req.Td`): the hold delays the voltage by $T_s/2$ on average and the new value comes one sample later. A delay removes phase ωT_d without changing the gain, so the scripts compute the margins with it (`P.InputDelay = Td` in `closed_loop.m`).

3 Pole placement

Gains from the poles (polynomial matching)

With $\hat x = x$, states $[\theta, \omega, e_I]$, the closed loop has $\Delta(s) = s^3 + a_2 s^2 + a_1 s + a_0$ with $a_2 = \frac{1+bk_2}{\tau}$, $a_1 = \frac{bk_1}{\tau}$, $a_0 = -\frac{bk_i}{\tau}$. Choosing the poles fixes $\Delta(s) = (s-p_1)(s-p_2)(s-p_3)$, hence

$$k_1 = \frac{\tau a_1}{b}, \qquad k_2 = \frac{\tau a_2 - 1}{b}, \qquad k_i = -\frac{\tau a_0}{b}, \qquad a_2 = -\sum p_i,\ a_1 = \sum_{i<j} p_i p_j,\ a_0 = -p_1 p_2 p_3.$$

Each gain appears in one coefficient only, so the gains exist and are unique. In the code `acker(Ai, Bi, p)` returns the same $K = [k_1, k_2, k_i]$.

Observer. Poles $-o_1, -o_2$ with $o_1 = \min(6 \max|p_i|, 250)$, $o_2 = 1.25\,o_1$ (250 rad/s $= 0.5/T_s$): the estimate must be faster than the loop, but the digital controller cannot go further. $L =$ `acker(A', C', p_obs)'`. **Separation:** the estimation error obeys $\dot\varepsilon = (A - LC)\varepsilon$, so the observer does not move the three controller poles, and the reference sees only those. **PID start:** the PID is the state feedback $K = [K_p, K_d, -K_i]$; its poles (slow one ≈ 0.36 rad/s) can be used as a starting point (`start_from_pid`).

4 The parameters and their role

Approach A writes the poles as a complex pair and a real pole: $p_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$, $\quad p_3 = -\lambda = -\alpha\zeta\omega_n$.

What each parameter does physically

parameter	physical meaning	typical value here
ω_n	speed and stiffness of the joint: $k_1 \approx \tau\omega_n^2/\lvert b\rvert$ volts per degree of error. Higher: faster, smaller error under friction ($e \approx \Delta u/k_1$), but more voltage on big errors, more encoder noise on u, less phase margin	50–65 rad/s
ζ	damping of the pair: low $\Rightarrow$ overshoot and ringing at the corners, larger M_s; $\to 1$ no oscillation, slower, larger k_2 (more noise from $\hat{\omega}$)	0.85–0.95
α (λ)	speed of the integral action: a residual error decays as $e^{-\lambda t}$, gone in $\approx 4/\lambda$; $k_i \approx \tau\omega_n^2\lambda/\lvert b\rvert$. Too close to the pair ($\alpha \gtrsim 0.2$) it removes phase at the crossover	$\lambda = 5$ rad/s
β	weight of the reference in the feedback: outside the loop, it changes only the zero of θ/r (section 5), not the poles, margins or load response	1 (paths), β^* (steps)
o_1, o_2	observer speed, fixed by the rule above	≤ 250 rad/s
K_{aw}	how fast the integral is unloaded during saturation	$-\overline{\lvert p\rvert}/k_i$
R	size of the test step of the requirements (section 6)	3° / 21°

Approach B (`design_B.m`) uses three real poles $-\lambda_1, -\lambda_2, -\lambda_3$: no oscillating mode, each pole moves on its own, slightly slower rise. Everything else (gains, observer, requirements, search) is the same. `tune_poles.m` moves ω_n, ζ, α with sliders for both motors; with *lock alpha* $\alpha = \lambda/(\zeta\omega_n)$ keeps the integral pole fixed while ω_n and ζ change.

5 The reference weight β

Reference to angle: one zero

$$\frac{\Theta(s)}{R(s)} = \frac{\beta a_1 s + a_0}{\Delta(s)}, \qquad z = -\frac{a_0}{\beta a_1}, \qquad \frac{\Theta(0)}{R(0)} = 1 \text{ for every } \beta.$$

A zero adds the derivative of the response: the step becomes faster and, if the zero is slow, it overshoots.

β	zero	step	use
$\beta = 1$	$\lvert z\rvert = a_0/a_1 \approx \lambda$, slower than every pole	overshoot (also with real poles); no lag on a path	**trajectories**
$\beta^* = \dfrac{a_0}{a_1\lambda}$	exactly on the real pole: cancels it	follows the pair only (A: classic second order, overshoot set by ζ)	**steps**
$\beta = 0$	none	as slow as the integral pole	not used

- β is outside the loop: poles, margins, M_s, chatter and the load response are the same for every β.
- The cancellation by β^* hides the real pole from the reference only; loads and friction still see it.
- On a path, $\beta < 1$ makes the feedback chase a reference smaller than the real one: the integral has to make up $(1-\beta)k_1\varphi$ continuously, so the tracking error grows. **For paths use $\beta = 1$.**
- In design A with β^*: $\beta^* = 1/(1+2\alpha\zeta^2)$ and $\Theta/R = \omega_n^2/(s^2 + 2\zeta\omega_n s + \omega_n^2)$.

6 The requirements

All in `utils/pp_setup.m` (section 1); `design_A.m` switches some of them with `beta_mode`. Each one is checked on **both motors**, the worst counts. They are measured on a reference step of R degrees simulated with the 3 ms delay (`closed_loop.m`).

The seven requirements

requirement	limit	meaning
overshoot	`OS_max` [%]	how far the step goes past R
settling 2 %	`ts_max` [s]	when the step stays within ±2% of R
peak voltage	`Vmax` = 6 V	with $\beta = 1$: $u(0^+) = k_1 R$, so in practice $k_1 \leq V_{max}/R$
phase margin	`PM_min` = 45°	robustness to the delay and to errors in k, τ
chatter	`chatter` [V]	voltage jump caused by one encoder count: $\max_{\omega \geq 100} \lvert C(j\omega)\rvert \cdot q$
slowest pole	`lam_min` [rad/s]	speed of the integral action
M_s	`Ms_max` = 1.6	peak of $\lvert 1/(1+L)\rvert$ with the delay: $1/M_s$ = distance of Nyquist from -1; $M_s \leq 1.6 \Rightarrow$ $GM \geq 8.5$ dB and $PM \geq 36°$ at all frequencies together

Choice of R: tests with the inverse kinematics

R is not a requirement by itself: only the peak-voltage check depends on it (overshoot and settling are relative to R). It must represent the largest step the controller really receives. With the inverse kinematics from home $C = (l, l)$, the centre of the square, the joint moves are:

target	φ (motor 3165)	θ (motor 3166)
side midpoint East / West	3.8° / 3.7°	−20.6° / +20.9°
side midpoint North / South	+20.9° / −20.6°	3.8° / 3.7°
corners SW, SE, NE, NW	−17.8, −15.9, 28.3, 23.1°	23.1, −17.8, −15.9, 28.3°

So the largest step the CNC has to make is $\approx 21°$ (start from a side midpoint, the new trajectory) and at most $\approx 28°$ (a corner). During the laps the error is a few degrees at most.

Two profiles: `beta_mode = 'one'` (paths) and `'star'` (steps)

requirement	$\beta = 1$ (paths)	β^* (steps)	why
R	3°	21°	paths: the largest error corrected linearly; steps: the real move (table above)
overshoot	off (`Inf`)	2 %	$\beta = 1$: the zero always gives overshoot, and on a CNC the approach to the path is not part of the work (excluded by `t_skip`). β^*: the pair decides it
settling	off (`Inf`)	0.2 s	a pure step never happens on the path; with $\beta = 1$ a fast integral pole leaves a pole–zero tail that the search would read as a violation
peak voltage	6 V	6 V	real saturation
phase margin	45°	45°	same loop for both β
chatter	0.5 V	0.5 V	encoder noise: in practice the cap on ω_n
slowest pole	5 rad/s	5 rad/s	see below
M_s	1.6	1.6	same loop for both β

Slowest pole. A residual error (friction changes sign at every corner) must disappear within one side of the square ($T/4 = 1$ s): $4/\lambda \le T/4 \Rightarrow \lambda \ge 16/T = 4$ rad/s. It must stay well below the pair: $\lambda \le 0.2\,\zeta\omega_n$ (≈ 10 rad/s with $\omega_n = 55$). Hence $\lambda_{min} = 5$ rad/s. It is not $1/t_s$: a first-order mode needs $\approx 4/\lambda$ to settle, and with β^* the real pole is hidden from the step anyway.

Note on $R = 21°$. With a pure step the voltage check gives $k_1 \lesssim 0.3\,\mathrm{V}/°$ ($\omega_n \lesssim$ 25–30 rad/s), a soft loop; on the robot the 100°/s rate limiter makes the real step gentler. Comparing the two profiles therefore compares two different controllers, not only two values of β; to see the effect of β alone keep the same poles (`do_search = false` or `check_design`).

7 The search

Cost minimised by `fminsearch` (`utils/pp_search.m`)

$$J(x) = \underbrace{\sum_{i=1}^{3} \left(\log|p_{(i)}(x)| - \log|p_{0(i)}|\right)^2}_{\text{stay close to the start}} + w \underbrace{\sum_{k=1}^{7} \max(0, g_k(x))^2}_{\text{only violated requirements}}, \qquad w = 10^4.$$

- x: A $= [\omega_n, \zeta, \alpha]$, B $= [\lambda_1, \lambda_2, \lambda_3]$, searched in logarithms (always positive); p_0 = start poles (PID or your values); poles compared sorted by speed.
- Normalised violations: $g =$ value/limit $- 1$ (upper limits), $g = 1 -$ value/limit (lower limits); $g \le 0$ met, $g = 0.1$ means 10 % beyond.
- Nelder–Mead: 4 trial points rolling downhill on J, no derivatives (overshoot and settling are not smooth), up to 4 restarts until every $g_k \le 10^{-3}$.
- The penalty is **soft**: if two requirements conflict, the result is the compromise with the smallest total violation and the table shows a `*`.
- The search **depends on** β: each candidate's step is simulated with the β of `beta_mode`, and `design_A.m` also changes R, overshoot and settling with it. M_s, phase margin, chatter and the slowest pole do not depend on β.
- Relaxing requirements does not push the poles faster (the search stays close to the start): to get a stiffer design start from your values (`start_from_pid = false`).

8 The trajectory: square with a step at the start

`square_gen_smooth` (MATLAB Function block `square trajectory`) gives $x, y, \dot{x}, \dot{y}, \ddot{x}, \ddot{y}$ of the tip; the inverse kinematics turns them into φ, θ and their derivatives.

- $0 \le t < t_{hold}$ (1.5 s): the reference is the start point Q, the midpoint of one side, held still (zero speed and acceleration, so the feedforward is zero). The robot starts at home = centre of the square, so this is a **step** for the loop, mostly on one motor (East/West → 3166, North/South → 3165).
- $t \ge t_{hold}$: laps counter-clockwise $Q \to 4$ corners $\to Q$, lap time $T = 4$ s; half sides at both ends, so all sides have the same speed. Each segment uses the quintic $\sigma = 10f^3 - 15f^4 + 6f^5$: zero speed and acceleration at both ends, the tip stops at every corner.
- A step is a straight line in *joint* space, not in the plane, and each joint has its own rate limiter: the approach to Q is curved. This does not matter for the evaluation (excluded by `t_skip`); for a straight approach, give the first t_{app} seconds a quintic from home to Q instead of the step.
- The rate limiters (100°/s) act on the positions only, while `dphi`, `ddphi` are not limited: the path must stay below 100°/s per joint (it reaches ≈ 80°/s with $T = 4$ s), otherwise position reference and feedforward disagree and the loop oscillates.

9 Evaluation with `validate_controllers.m`

option	value	meaning
`N_pid`	100	derivative filter of the PID block (as in `CNC_pid_ff.slx`); only for PID runs
`enc_res`	360/4096	one encoder count [deg]: converts the controller gain into the noise on u
`k_var`, `tau_var`	0.2, [0.021 0.045]	range of k and τ for "M_s worst" (covers no load and load)
`Vmax`	6	used only if the run file has no `Vmax`
`t_skip`	$t_{hold} + T = 5.5$ s	start of the evaluated data

Why $t_{skip} = t_{hold} + T$. After t_{hold} the step is over; one more lap leaves time for the integral transient ($4/\lambda = 0.8$ s with $\lambda = 5$). Every run is then evaluated on *complete laps* only, so the RMS does not depend on where the run stops: set the stop time to $t_{skip} + nT$ (e.g. 25.5 s). If T or t_{hold} change, use the same rule; keep $4/\lambda < T$.
What to read. *Contour error* = distance of the tip from the path (the shape of the drawing); *tracking error* = distance from the point at the same time (it also contains time lag); joint error RMS and max; peak $|u|$ and time in saturation. The model table (M_s, margins, noise) is computed without the delay: it is lower than the M_s of the design scripts.

10 Approaches A and B in one table

	A: pair + real pole	B: three real poles
search variables	ω_n, ζ, α	$\lambda_1, \lambda_2, \lambda_3$
β for steps / paths	$\beta^* = 1/(1+2\alpha\zeta^2)$ / 1	$\beta^* = a_0/(a_1\lambda_1)$ / 1
step with β^*	second order, overshoot set by ζ (2 % at $\zeta \approx 0.78$)	no overshoot, slower rise
weak point	exact cancellation needed for steps	the same; slower

11 Disturbance modelling (simulation)

`parameters.m`, section 5: a "real" plant different from the model

The controller uses k, τ = `*_load`; the simulated motor should use `*_plant`, so model and plant differ as on the bench. With `use_real_plant = true`: $k = 1.77/1.74$, $\tau = 0.0428/0.0444$ s, delay $T_d = 3.5$ ms, Coulomb friction $V_c = 0.12/0.14$ V (motor 3165/3166), $w_\varepsilon = 0.02$ rad/s; with `false` the plant equals the model and $T_d = V_c = 0$. The design scripts load these variables (`pp_simulink.m` runs `parameters.m`).

- **Friction in the plant:** in each motor subsystem subtract $V_c \tanh(\omega/w_\varepsilon)$ from the voltage before $k/(\tau s + 1)$, with ω the output of that block (no algebraic loop: the block is strictly proper). Use `k_/tau_xxxx_plant` in the transfer function and the delay T_d at the motor input.
- **Friction compensation in the controller** (section 2): $V_c \tanh(\dot\varphi/w_\varepsilon)$ from the reference speed. Keep its value separate from the plant friction (e.g. `Vcff_xxxx`), so that the robot also gets it when `use_real_plant = false` and the simulation is not optimistically exact.
- **Other effects to add** once measured on the bench: encoder quantisation (`Quantizer`, $q = 0.088°$), electrical noise (band-limited noise in the load input d), backlash. Validate by running the same path in simulation and on the robot and comparing them with `validate_controllers.m`.

12 Using more of the plant: voltage and stiffness

Where the voltage goes on the path

$u = u_{ff} + u_{fb}$. $u_{ff} = (\dot{r} + \tau\ddot{r})/b$ is set by the trajectory: with $T = 4\,\mathrm{s}$ the joints reach $\approx 80°/\mathrm{s}$ and $\approx 300°/\mathrm{s}^2$, i.e. $\approx 1\,\mathrm{V} + 0.14\,\mathrm{V}$; at 6 V a joint could turn at $\approx 490°/\mathrm{s}$. u_{fb} supplies only what the feedforward misses ($\Delta u \approx 0.1\,\mathrm{V}$ of friction) with an error $e \approx \Delta u/k_1$. A larger ω_n *does not raise the amplitude of* u: it gives the same Δu with a smaller error. More voltage along the whole path comes only from a faster path (shorter T, after raising the rate limiters) or from a feedforward that also contains the friction.

limit on ω_n (motor 3165, $\zeta = 0.95$)	ω_n max [rad/s]
observer 4–6 times faster than the controller (capped at 250 rad/s)	42–62
chatter $\leq 0.5\,\mathrm{V/count}$	60–65
$M_s \leq 1.6$ (with the 3 ms delay)	60–65
phase margin $\geq 45°$ ($\omega_c T_d$ of phase lost)	75–80
sampling ($\omega_c \lesssim \omega_s/20$)	150

The practical range is $\omega_n \approx 55$–$65\,\mathrm{rad/s}$, about twice the mechanical pole $1/\tau$. On the robot raise ω_n in steps and stop when the motor buzzes, the noise on u exceeds $\approx 0.5\,\mathrm{V}$ or M_s exceeds ≈ 1.6. Other limits: 1 A continuous current ($R_m = 2.6\,\Omega$), backlash and the linkage at high speed.